

Tensor Memory Layout Conversion

<https://gemini.google.com/app/dc00a564eb2c33c2>

User prompt: use recursive, iterative, or by index recalculation, solve Problem Constructing algorithms for high-dimensional data is often made more difficult by how data is actually laid out in memory. Operations must be translated to work on linearized representations of the real data, ideally while preserving as much locality and parallelism as possible. In this homework, you are tasked with implementing arbitrary-dimensional tensor reordering, specifically from Fortran style (column major) ordering to C style (row major) ordering. This is a generalization of the usual matrix transposition problem and the bit reversal seen in FFT algorithms.

Response: To convert an arbitrary N -dimensional tensor from Fortran-style (column-major) memory layout to C-style (row-major) memory layout, we need to reorder its flat 1D array representation.

In **Fortran (column-major)** ordering, the first dimension varies fastest.

In **C (row-major)** ordering, the last dimension varies fastest.

Mathematical Mapping

Let a tensor have shape $\mathbf{S} = [d_0, d_1, \dots, d_{N-1}]$.

For a multi-dimensional index tuple $(i_0, i_1, \dots, i_{N-1})$ where $0 \leq i_k < d_k$:

1. Fortran-major (Column-major) Flat Index (I_F):

$$I_F = i_0 + i_1 \cdot d_0 + i_2 \cdot (d_0 \cdot d_1) + \dots + i_{N-1} \cdot \prod_{k=0}^{N-2} d_k$$

2. C-major (Row-major) Flat Index (I_C):

$$I_C = i_{N-1} + i_{N-2} \cdot d_{N-1} + i_{N-3} \cdot (d_{N-1} \cdot d_{N-2}) + \dots + i_0 \cdot \prod_{k=1}^{N-1} d_k$$

Implementation Approaches in Python

Approach 1: By Index Recalculation

Map every flat index in the Fortran array to its corresponding multi-dimensional coordinate tuple, then convert that tuple into the corresponding flat C index.

```
def f2c_index_recalculation(fortran_arr, shape):
    """
    Converts a column-major flat list/array to a row-major flat list via index conversion.
    """
    total_elements = 1
    for dim in shape:
        total_elements *= dim

    # Precalculate Fortran strides: stride_F[k] = prod(shape[0...k-1])
    strides_f = [1] * len(shape)
    for k in range(1, len(shape)):
        strides_f[k] = strides_f[k - 1] * shape[k - 1]

    # Precalculate C strides: stride_C[k] = prod(shape[k+1...N-1])
    strides_c = [1] * len(shape)
    for k in range(len(shape) - 2, -1, -1):
        strides_c[k] = strides_c[k + 1] * shape[k + 1]

    c_arr = [None] * total_elements
```

```

for flat_f in range(total_elements):
    # 1. Unravel flat_f to multi-dimensional index tuple (i_0, i_1, ..., i_{N-1})
    temp = flat_f
    coords = [0] * len(shape)
    for k in range(len(shape) - 1, -1, -1):
        coords[k] = temp // strides_f[k]
        temp %= strides_f[k]

    # 2. Ravel multi-dimensional index tuple to flat_c
    flat_c = sum(coords[k] * strides_c[k] for k in range(len(shape)))

    c_arr[flat_c] = fortran_arr[flat_f]

return c_arr

```

Approach 2: Iterative Strided Loop

Avoid unraveling indices for every single element by maintaining multi-dimensional coordinate counters and updating the flat destination offset incrementally.

```

def f2c_iterative(fortran_arr, shape):
    """
    Converts Fortran order to C order iteratively maintaining current indices and offsets.
    """
    ndim = len(shape)
    if ndim == 0:
        return list(fortran_arr)

    total_elements = len(fortran_arr)

    # Precalculate C strides
    strides_c = [1] * ndim
    for k in range(ndim - 2, -1, -1):
        strides_c[k] = strides_c[k + 1] * shape[k + 1]

    c_arr = [None] * total_elements

    coords = [0] * ndim
    curr_c_idx = 0

    for flat_f in range(total_elements):
        c_arr[curr_c_idx] = fortran_arr[flat_f]

        # Increment Fortran coordinates (dim 0 varies fastest)
        for dim in range(ndim):
            coords[dim] += 1
            curr_c_idx += strides_c[dim] # Update C offset for step in dim

            if coords[dim] < shape[dim]:
                break
            else:
                # Reset dimension and subtract accumulated stride effect
                coords[dim] = 0
                curr_c_idx -= shape[dim] * strides_c[dim]

    return c_arr

```

Approach 3: Recursive Decomposition

Decompose the tensor along dimensions to maximize cache efficiency and simplify multi-dimensional indexing.

```

def f2c_recursive(fortran_arr, shape):
    """
    Recursively converts Fortran-ordered flat array to C-ordered flat array.
    """
    total_elements = len(fortran_arr)
    c_arr = [None] * total_elements

    # Compute strides for both layouts

```

```
strides_f = [1] * len(shape)
for k in range(1, len(shape)):
    strides_f[k] = strides_f[k - 1] * shape[k - 1]

strides_c = [1] * len(shape)
for k in range(len(shape) - 2, -1, -1):
    strides_c[k] = strides_c[k + 1] * shape[k + 1]

def recurse(dim, offset_f, offset_c):
    if dim == len(shape):
        c_arr[offset_c] = fortran_arr[offset_f]
        return

    for i in range(shape[dim]):
        recurse(
            dim + 1,
            offset_f + i * strides_f[dim],
            offset_c + i * strides_c[dim],
        )

recurse(dim=0, offset_f=0, offset_c=0)
return c_arr
```

Complexity & Comparison

Approach	Time Complexity	Memory Locality	Key Advantage
Index Recalculation	$\mathcal{O}(N \cdot \prod d_i)$	Poor	Direct, stateless mapping for parallel loops
Iterative	$\mathcal{O}(\prod d_i)$	Good	Minimal overhead, no recursion stack
Recursive	$\mathcal{O}(\prod d_i)$	Superior	Easily adaptable to cache-oblivious block reordering

User prompt: use

Response:

User prompt: use c++

Response: Here are the complete C++ implementations for arbitrary N -dimensional Fortran-to-C layout conversion using **Index Recalculation**, **Iterative Strided Loop**, and **Recursive Traversal**.

Prerequisites & Setup

```
#include <iostream>
#include <vector>
#include <numeric>

// Helper to calculate total elements: product of dimensions
size_t get_total_elements(const std::vector<size_t>& shape) {
    size_t total = 1;
    for (size_t dim : shape) total *= dim;
    return total;
}

// Compute strides for Fortran ordering (column-major)
std::vector<size_t> compute_fortran_strides(const std::vector<size_t>& shape) {
    size_t ndim = shape.size();
    std::vector<size_t> strides(ndim, 1);
    for (int i = ndim - 2; i >= 0; --i)
        strides[i] = strides[i + 1] * shape[i + 1];
    return strides;
}
```

```

    std::vector<size_t> strides(ndim, 1);
    for (size_t k = 1; k < ndim; ++k) {
        strides[k] = strides[k - 1] * shape[k - 1];
    }
    return strides;
}

// Compute strides for C ordering (row-major)
std::vector<size_t> compute_c_strides(const std::vector<size_t>& shape) {
    size_t ndim = shape.size();
    std::vector<size_t> strides(ndim, 1);
    if (ndim > 0) {
        for (int k = static_cast<int>(ndim) - 2; k >= 0; --k) {
            strides[k] = strides[k + 1] * shape[k + 1];
        }
    }
    return strides;
}

```

Method 1: By Index Recalculation

In this approach, each 1D Fortran index I_F is unravelled into multi-dimensional coordinates $(i_0, i_1, \dots, i_{N-1})$, which are then linearly combined with C strides to determine I_C .

- **Key advantage:** Every element evaluation is completely independent, making it trivial to parallelize with `#pragma omp parallel for`.

```

template <typename T>
std::vector<T> f2c_index_recalculation(const std::vector<T>& src,
                                       const std::vector<size_t>& shape) {
    size_t total_elements = src.size();
    size_t ndim = shape.size();

    std::vector<size_t> strides_f = compute_fortran_strides(shape);
    std::vector<size_t> strides_c = compute_c_strides(shape);
    std::vector<T> dst(total_elements);

    for (size_t flat_f = 0; flat_f < total_elements; ++flat_f) {
        size_t temp = flat_f;
        size_t flat_c = 0;

        // Unravel flat_f to coordinates and compute flat_c
        for (int k = static_cast<int>(ndim) - 1; k >= 0; --k) {
            size_t coord = temp / strides_f[k];
            temp %= strides_f[k];
            flat_c += coord * strides_c[k];
        }

        dst[flat_c] = src[flat_f];
    }

    return dst;
}

```

Method 2: Iterative Strided Loop

Instead of recomputing modulo arithmetic for every element, this maintains multi-dimensional coordinate counters and updates the target row-major offset incrementally.

```

template <typename T>
std::vector<T> f2c_iterative(const std::vector<T>& src,
                             const std::vector<size_t>& shape) {
    size_t total_elements = src.size();
    size_t ndim = shape.size();

    std::vector<size_t> strides_c = compute_c_strides(shape);
    std::vector<T> dst(total_elements);

```

```

std::vector<size_t> coords(ndim, 0);
size_t curr_c_idx = 0;

for (size_t flat_f = 0; flat_f < total_elements; ++flat_f) {
    dst[curr_c_idx] = src[flat_f];

    // Carry-over increment for multi-dimensional index
    for (size_t dim = 0; dim < ndim; ++dim) {
        coords[dim]++;
        curr_c_idx += strides_c[dim];

        if (coords[dim] < shape[dim]) {
            break;
        } else {
            // Reset dimension counter and subtract accumulated offset
            coords[dim] = 0;
            curr_c_idx -= shape[dim] * strides_c[dim];
        }
    }
}

return dst;
}

```

Method 3: Recursive Decomposition

Recursively branches through dimensions down to the base case (single item or base dimension loop).

```

template <typename T>
void f2c_recursive_helper(const std::vector<T>& src, std::vector<T>& dst,
                        const std::vector<size_t>& shape,
                        const std::vector<size_t>& strides_f,
                        const std::vector<size_t>& strides_c,
                        size_t dim, size_t offset_f, size_t offset_c) {
    if (dim == shape.size()) {
        dst[offset_c] = src[offset_f];
        return;
    }

    size_t d_f = strides_f[dim];
    size_t d_c = strides_c[dim];
    size_t extent = shape[dim];

    for (size_t i = 0; i < extent; ++i) {
        f2c_recursive_helper(src, dst, shape, strides_f, strides_c,
                            dim + 1, offset_f + i * d_f, offset_c + i * d_c);
    }
}

template <typename T>
std::vector<T> f2c_recursive(const std::vector<T>& src,
                            const std::vector<size_t>& shape) {
    std::vector<T> dst(src.size());
    std::vector<size_t> strides_f = compute_fortran_strides(shape);
    std::vector<size_t> strides_c = compute_c_strides(shape);

    f2c_recursive_helper(src, dst, shape, strides_f, strides_c, 0, 0, 0);
    return dst;
}

```

Verification Driver Example

```

int main() {
    // 3D Tensor: shape [2, 3, 4]
    std::vector<size_t> shape = {2, 3, 4};
    size_t total = get_total_elements(shape);

    // Initialize source array with 0, 1, 2, ..., total - 1
    std::vector<int> src(total);
    std::iota(src.begin(), src.end(), 0);
}

```

```

auto res_index = f2c_index_recalculation(src, shape);
auto res_iter  = f2c_iterative(src, shape);
auto res_recur = f2c_recursive(src, shape);

// Verify all methods match
bool ok = (res_index == res_iter) && (res_iter == res_recur);
std::cout << "All methods produced identical results: "
           << (ok ? "YES" : "NO") << std::endl;

return 0;
}

```

User prompt: Can you show an OpenMP multi-threaded and cache-blocked implementation of Fortran to C layout reordering in C++?

Response: For high-dimensional tensors, naive element-by-element transposition suffers from poor cache locality because contiguous accesses in the source array translate to widely spaced non-contiguous strides in the destination array.

By combining **OpenMP multi-threading** (parallelizing outer dimensions) and **Cache Blocking / Tiling** (processing sub-tensors that fit entirely within L1/L2 cache), we achieve near-peak memory bandwidth.

Key Optimization Concepts

1. **Outer-Dimension Parallelism:** Parallelize over the outer dimensions using OpenMP (`#pragma omp parallel for`) so that threads work on completely disjoint regions of memory without data races.
2. **N-Dimensional Tiling/Blocking:** Instead of iterating through entire dimensions sequentially, process each dimension in blocks of size B_k (e.g., $B = 32$ or 64).
3. **Sequential Inner Store:** For the innermost block loops, iterate through contiguous chunks to maximize CPU vectorization and store buffer utilization.

High-Performance OpenMP + Cache-Blocked C++ Implementation

```

#include <iostream>
#include <vector>
#include <algorithm>
#include <numeric>
#include <omp.h>

// Helper to compute strides
std::vector<size_t> compute_fortran_strides(const std::vector<size_t>& shape) {
    size_t ndim = shape.size();
    std::vector<size_t> strides(ndim, 1);
    for (size_t k = 1; k < ndim; ++k) {
        strides[k] = strides[k - 1] * shape[k - 1];
    }
    return strides;
}

std::vector<size_t> compute_c_strides(const std::vector<size_t>& shape) {
    size_t ndim = shape.size();
    std::vector<size_t> strides(ndim, 1);
    if (ndim > 0) {
        for (int k = static_cast<int>(ndim) - 2; k >= 0; --k) {
            strides[k] = strides[k + 1] * shape[k + 1];
        }
    }
    return strides;
}

/**

```

```

* Recursive helper to copy a single tile/block across dimensions.
*/
template <typename T>
void process_block_recursive(
    const T* __restrict src,
    T* __restrict dst,
    const std::vector<size_t>& shape,
    const std::vector<size_t>& strides_f,
    const std::vector<size_t>& strides_c,
    const std::vector<size_t>& block_starts,
    const std::vector<size_t>& block_sizes,
    size_t dim,
    size_t offset_f,
    size_t offset_c)
{
    // Base Case: copy elements inside the current block
    if (dim == shape.size()) {
        dst[offset_c] = src[offset_f];
        return;
    }

    size_t start = block_starts[dim];
    size_t end = start + block_sizes[dim];
    size_t stride_f = strides_f[dim];
    size_t stride_c = strides_c[dim];

    for (size_t i = start; i < end; ++i) {
        process_block_recursive(
            src, dst, shape, strides_f, strides_c,
            block_starts, block_sizes,
            dim + 1,
            offset_f + i * stride_f,
            offset_c + i * stride_c
        );
    }
}

/**
 * Parallel Cache-Blocked Fortran-to-C Transposition
 *
 * @param src Source tensor (Fortran-ordered)
 * @param shape Tensor dimensions
 * @param block_size Cache block size per dimension (default 32)
 */
template <typename T>
std::vector<T> f2c_parallel_blocked(
    const std::vector<T>& src,
    const std::vector<size_t>& shape,
    size_t block_size = 32)
{
    size_t ndim = shape.size();
    size_t total_elements = src.size();
    std::vector<T> dst(total_elements);

    if (ndim == 0) return dst;

    std::vector<size_t> strides_f = compute_fortran_strides(shape);
    std::vector<size_t> strides_c = compute_c_strides(shape);

    // Compute total number of blocks per dimension
    std::vector<size_t> num_blocks(ndim);
    size_t total_outer_blocks = 1;

    for (size_t k = 0; k < ndim; ++k) {
        num_blocks[k] = (shape[k] + block_size - 1) / block_size;
        total_outer_blocks *= num_blocks[k];
    }

    const T* src_ptr = src.data();
    T* dst_ptr = dst.data();

```

```

// Parallelize over the flattened block space
#pragma omp parallel for schedule(dynamic)
for (size_t block_idx = 0; block_idx < total_outer_blocks; ++block_idx) {
    // Decode block_idx into per-dimension block coordinates
    std::vector<size_t> b_starts(ndim);
    std::vector<size_t> b_sizes(ndim);
    size_t temp = block_idx;

    for (int k = static_cast<int>(ndim) - 1; k >= 0; --k) {
        size_t b_coord = temp % num_blocks[k];
        temp /= num_blocks[k];

        b_starts[k] = b_coord * block_size;
        b_sizes[k] = std::min(block_size, shape[k] - b_starts[k]);
    }

    // Process this sub-tensor block in L1/L2 cache
    process_block_recursive(
        src_ptr, dst_ptr, shape,
        strides_f, strides_c,
        b_starts, b_sizes,
        0, 0, 0
    );
}

return dst;
}

```

Verification and Benchmark Harness

Compile with OpenMP enabled:

```

g++ -O3 -fopenmp -march=native main.cpp -o f2c_benchmark

#include <chrono>

int main() {
    // 4D Tensor: [64, 64, 64, 64] -> ~16.7 Million float elements (~67 MB)
    std::vector<size_t> shape = {64, 64, 64, 64};
    size_t total_elements = 1;
    for (size_t d : shape) total_elements *= d;

    std::vector<float> src(total_elements);
    std::iota(src.begin(), src.end(), 0.0f);

    std::cout << "Tensor Shape: [64, 64, 64, 64] ("
        << (total_elements * sizeof(float)) / (1024 * 1024)
        << " MB)\n";
    std::cout << "OpenMP Threads: " << omp_get_max_threads() << "\n\n";

    // Benchmark Parallel Blocked Execution
    auto start = std::chrono::high_resolution_clock::now();

    // Block size 32 (32x32 floats fit inside L1 cache)
    auto dst = f2c_parallel_blocked(src, shape, 32);

    auto end = std::chrono::high_resolution_clock::now();
    double ms = std::chrono::duration<double, std::milli>(end - start).count();

    std::cout << "Execution Time: " << ms << " ms\n";
    std::cout << "Effective Bandwidth: "
        << (2.0 * total_elements * sizeof(float) / (ms * 1e-3)) / 1e9
        << " GB/s\n";

    return 0;
}

```

Performance Notes

- 1. **Tile Size Selection:** Choosing a block size of 16 to 64 ensures that B^N sub-blocks fit within L1/L2 caches, preventing cache-line thrashing during transpose operations.
- 2. **Keyword Optimization:** Using `__restrict` pointers hints to the compiler that `src` and `dst` do not overlap, enabling auto-vectorization within the inner loop blocks.
- 3. **Dynamic Scheduling:** `#pragma omp parallel for schedule(dynamic)` handles edge tile boundary imbalance across threads automatically.

User prompt: Which algorithm

Response: The optimal choice depends on your specific constraints regarding **performance**, **complexity**, and **scalability**:

Comparison of Algorithms

Feature	Index Recalculation	Iterative Strided Loop	Recursive Traversal	Parallel Cache-Blocked (Recommended)
Primary Mechanism	Modulo/division per element	Increment and carry-over counters	Depth-first dimensional tree search	Dimensional tiling/blocking + OpenMP threads
**Time				

User prompt: Which algorithm is suitable to run in parellel?

Response: The two algorithms suitable for parallel execution are **Index Recalculation** and **Cache-Blocked (Recursive or Iterative)**, but they suit different contexts:

1. Index Recalculation (Best for GPUs / Embarrassingly Parallel Workloads)

Why It Works in Parallel

Every element's source and destination indices are derived purely from its flat 1D index using stateless math (modulo and integer division). There is zero shared state, no inter-thread dependency, and no data race.

Best Used For

- **GPUs (CUDA / SYCL / OpenCL):** High-throughput execution where thousands of simple threads compute their own indices simultaneously.
- **Simple CPU OpenMP Parallelization:** Adding a basic `#pragma omp parallel for` over the flat loop array without setting up complex multi-dimensional loops.

Downside

- **Poor Cache Locality:** Coalesced reads from Fortran order lead to completely non-contiguous writes in C order across cache lines.

```
// Embarrassingly parallel over all elements
#pragma omp parallel for schedule(static)
for (size_t flat_f = 0; flat_f < total_elements; ++flat_f) {
    size_t temp = flat_f;
```

```

size_t flat_c = 0;
for (int k = static_cast<int>(ndim) - 1; k >= 0; --k) {
    size_t coord = temp / strides_f[k];
    temp %= strides_f[k];
    flat_c += coord * strides_c[k];
}
dst[flat_c] = src[flat_f];
}

```

2. Parallel Cache-Blocked / Tiled Decomposition (*Best for Multi-Core CPUs*)

Why It Works in Parallel

Instead of parallelizing element-by-element, you divide the multi-dimensional tensor into non-overlapping sub-tensor blocks (tiles). Each OpenMP thread owns a distinct tile, eliminating write conflicts and race conditions.

Best Used For

- **Multi-Core CPUs (OpenMP / C++ Threads):** Maximizes L1/L2 cache hit rates and memory bandwidth saturation on modern hardware architectures.

Advantage

- **Superior Locality:** Avoids cache thrashing by ensuring all data needed for a block stays inside the CPU cache while performing the transposition.

```

// Parallelize over independent sub-tensor tiles
#pragma omp parallel for schedule(dynamic)
for (size_t block_idx = 0; block_idx < total_blocks; ++block_idx) {
    // Process block recursively or iteratively within L1/L2 cache
    process_tile(block_idx, src, dst);
}

```

Summary Recommendation

- **For CPUs:** Use **Parallel Cache-Blocked** (highest memory bandwidth, lowest cache miss rate).
- **For GPUs:** Use **Index Recalculation** (fits SIMD/SIMT architectures with minimal branching complexity).
- **Avoid:** The single-threaded **Iterative Strided Loop**, as its stateful carry-over counter creates sequential dependencies across iterations.

User prompt: what experiments needs to be done for this requirement: Problem Constructing algorithms for high-dimensional data is often made more difficult by how data is actually laid out in memory. Operations must be translated to work on linearized representations of the real data, ideally while preserving as much locality and parallelism as possible. In this homework, you are tasked with implementing arbitrary-dimensional tensor reordering, specifically from Fortran style (column major) ordering to C style (row major) ordering. This is a generalization of the usual matrix transposition problem and the bit reversal seen in FFT algorithms. Part 0 Pick one of the 3 algorithm approaches (recursive, iterative, or by index recalculation) by signing up to the respective slot, and think about how to complete this problem using that approach (you will eventually be required to implement all 3). You do not need to figure out the full algorithm, but consider the building blocks and/or overall strategy. You will discuss your findings with other students during the recitation. Part 1 Following the discussion during recitation, make a plan outlining your next steps. This includes, for instance, a brief, high-level description of all 3 algorithm approaches. We expect just a few sentences, not a design document, but you should clearly distill and explain your thoughts and plans. Part 2 Pick any programming language that supports shared memory parallelism well and implement the reordering using all 3 discussed approaches. If the language supports multidimensional arrays (e.g. Python n-d Numpy arrays), implement the recursive approach twice using both 1-d and n-d data types representing the same operation. Do not use library calls that implement any kind of matrix/tensor reordering (e.g.

`numpy.transpose`, `numpy.permute_dims`). You may use generative AI to assist in coding, though make sure that the code both runs and is actually correct. Machines. Pick one machine with a large number of cores from the following list: • An ECE number cluster machine: `ece0XX.ece.local.cmu.edu` <https://userguide.its.cit.cmu.edu/research-computing/computer-clusters/> • An AWS instance of your choice with multiple vCPU <https://aws.amazon.com/ec2/instance-types/> Links to an external site. • PSC Bridges RM <https://www.psc.edu/bridges/user-guide/running-jobs> Links to an external site. Algorithm and implementation. A tensor is an n -dimensional data cube. To store it in memory a linearization scheme needs to be decided upon. For instance, the C language utilizes row major storage: `double A[K][M][N]` and `double B[K*M*N]` have the relationship that for `(A == &B)` it holds that `A[i][j][k] = B[M*N*i + N*j + k]` for `(0 <= i < K) && (0 <= j < M) && (0 <= m < N)` The rotated cube will be represented by `double C[N][M][K]` and for `(C == &B)` it holds that `C[k][j][i] = B[M*K*k + K*j + i]` for `(0 <= i < K) && (0 <= j < M) && (0 <= m < N)` Note that for an odd rank (in this case rank 3) the middle dimension is unchanged while for even rank (e.g., rank 2, i.e., matrix transposition) all dimensions are changed. Implement the tensor reordering as cleanly and as concisely as possible. Utilize recursion or loops. Note that doing this may be hard for arbitrary rank n . Do not overstretch by trying to implement the problem efficiently right from the start. Once you have a correct implementation, spend minimal effort to parallelize one of the implementations for the targeted multicore processor. Note that the processor count gives you a hard limit of potential speed-up and suggests how to parallelize. Be careful if you are using Python, as the global interpreter lock (GIL) can be deceptive and prevents parallel code from running on multiple cores if not taken into account. Measurements. Create a linear data cube of 232 (1 billion) char elements (4 GB). Interpret it as “square” rank n tensor in all ways that the divisibility allows. For a chosen p way multicore processor, execute your parallel implementation on 1 to p processors/cores (i.e., on a quadcore run on 1, 2, 3, and 4 cores). • Measure runtime in seconds as function of problem size, using the tools the language provides (`gettimeofday()` in C/C++, `time` in Fortran, `System.currentTimeMillis()` in Java, `tic/toc` in Matlab, `time.time()` in Python, `tic()/toc()` in Julia, etc.). • Derive the memory traffic (number bytes loaded and stored) for your tensor reorder implementations as a function of tensor rank. Pick any programming language and implement a parallel reorder program and assess its memory performance and parallel speed-up and roofline behavior. The data cube shall be dense and complete, i.e., the number of elements is the product of the number of elements in all dimensions. Results presentation. Create a plot summarizing the measured bandwidth in GB/s as a function of the tensor rank varying across the number of cores. Plot one line per tensor rank, with the X axis being the number of cores used, and the y axis the measured bandwidth in GB/s. Use our provided Excel template for your plot.

Response: To satisfy the assignment's **Measurements** and **Results presentation** requirements, you need to conduct a structured set of performance benchmarking experiments on your target multi-core machine.

Here is the exact breakdown of the experiments you must perform, how to configure the tensor shapes, and how to record your measurements.

1. Dataset & Tensor Shapes Experiment Setup

- **Data Array Size:** Allocate a contiguous array of exactly 2^{30} (approx. 1.07×10^9) or 2^{32} char elements (**1 GB** or **4 GB** in memory).
- **Shape Selection ("Square" Rank N Tensors):** Interpret the total element count as a set of square or nearly-square N -dimensional tensors across different ranks N where the dimensions multiply to your total element count:
 - **Rank 2:** $2^{15} \times 2^{15}$ (or $2^{16} \times 2^{16}$ for 4 GB)
 - **Rank 3:** $1024 \times 1024 \times 1024$ (for 1 GB) or $1600 \times 1600 \times 1680$
 - **Rank 4:** $256 \times 256 \times 256 \times 256$ (for 4 GB)
 - **Rank 5:** $64 \times 64 \times 64 \times 64 \times 16$
 - **Higher Ranks ($N \geq 6$):** e.g., Rank 8 (16^8), Rank 10, or Rank 32 (2^{32}).

2. Core Scaling Experiment (Strong Scaling)

For each selected **Tensor Rank** N , execute your parallelized reorder algorithm across varying core counts ($1, 2, 3, \dots, p$) up to the maximum physical/virtual core count p on your chosen machine (AWS instance, PSC Bridges RM, or CMU ECE cluster).

Variables to Test:

- **X-Axis:** Core Count ($1 \rightarrow p$)
- **Y-Axis:** Measured Bandwidth (GB/s)
- **Series (Lines):** One line per Tensor Rank N

3. Measurements & Formulas to Record

For every combination of (**Tensor Rank** N , **Core Count** c):

A. Execution Time (T)

Measure wall-clock execution time in seconds (e.g., using `omp_get_wtime()` in C++ or `std::chrono`) excluding memory allocation or initialization time. Run 3–5 trials per data point and take the minimum or average time.

B. Theoretical Memory Traffic (M)

For a data array of E elements (`char` = 1 byte):

- **Bytes Read:** E bytes
- **Bytes Written:** E bytes
- **Total Memory Traffic:** $M = 2 \times E$ bytes (or $2 \times 4 \text{ GB} = 8 \text{ GB}$ transferred total).

C. Measured Effective Memory Bandwidth (BW)

$$\text{Bandwidth (GB/s)} = \frac{2 \times \text{Data Size in GB}}{\text{Execution Time } T \text{ (seconds)}}$$

Example: For a 4 GB array ($M = 8 \text{ GB}$ transferred) taking 0.20 seconds on 8 cores:

$$\text{Bandwidth} = \frac{8 \text{ GB}}{0.20 \text{ s}} = 40.0 \text{ GB/s}$$

4. Implementation Variants Experiment (Part 2 Verification)

Before running the full parallel scaling matrix, run a sanity check comparing all three implemented approaches to confirm correctness and baseline performance:

1. **Index Recalculation Implementation** (Sequential vs. Parallel)
2. **Iterative Strided Loop Implementation** (Sequential vs. Parallel)
3. **Recursive Decomposition Implementation**
 - *If using Python/NumPy:* Compare 1D flat array indexing vs. N -D native array indexing.

5. Expected Plot Layout (Excel Template Submission)

Your final result plot required for Part 2 / Submission will look like this:

- **Plot Type:** Line Chart (X-Y Scatter or Line Plot)
- **X-Axis:** Number of Cores used ($1, 2, 4, 8, \dots, p$)
- **Y-Axis:** Effective Bandwidth in **GB/s**
- **Series:** One line per rank (e.g., Rank 2, Rank 3, Rank 4, Rank 8, Rank 16)

Key Performance Insight to Look For: Lower ranks (like Rank 2 or 3) often saturate system memory bandwidth quickly, whereas very high ranks (like Rank 16 or 32) suffer from high stride computation overhead and cache line thrashing, resulting in lower total GB/s throughput even as core count increases.

User prompt: only apply parallel in Index Recalculation Implementation, explain experiments and working steps again

Response: When applying parallelism **strictly to the Index Recalculation approach**, your workflow and experimental plan simplify significantly. Because Index Recalculation is **stateless and embarrassingly parallel** (each 1D element's target index is computed independently), it maps directly to multi-core processing without any synchronization or data races.

Phase 1: Implementation & Working Steps

1. Implement All 3 Approaches Sequentially (Correctness Phase):

- **Index Recalculation:** Unravel flat Fortran index $I_F \rightarrow (i_0, \dots, i_{N-1}) \rightarrow$ compute flat C index I_C .
- **Iterative Strided Loop:** Update multi-dimensional coordinate counters sequentially with carry-over checks.
- **Recursive Traversal:** Recursively step through dimensions down to individual element assignments.
- **Validation:** Run all three on small tensors (e.g., $2 \times 3 \times 4$) and verify that `res_index == res_iterative == res_recursive`.

2. Parallelize **Only** Index Recalculation: Add OpenMP loop-level parallelism directly over the outer flat loop of the Index Recalculation function:

```
#pragma omp parallel for schedule(static) num_threads(num_cores)
for (size_t flat_f = 0; flat_f < total_elements; ++flat_f) {
    size_t temp = flat_f;
    size_t flat_c = 0;

    for (int k = static_cast<int>(ndim) - 1; k >= 0; --k) {
        size_t coord = temp / strides_f[k];
        temp %= strides_f[k];
        flat_c += coord * strides_c[k];
    }
    dst[flat_c] = src[flat_f];
}
```

Phase 2: Experimental Setup & Execution

1. Target Machine Selection

Choose your target multi-core machine (e.g., an AWS EC2 instance, CMU ECE cluster machine, or PSC Bridges RM) with p CPU cores. Note p as your max core limit.

2. Tensor Shape Configurations (Data Size = 2^{32} char elements / 4 GB)

Allocate a flat char buffer of $E = 2^{32} \approx 4.29 \times 10^9$ elements (4 GB). Shape it into "square" N -dimensional tensors across varying ranks:

- **Rank 2:** 65536×65536
- **Rank 4:** $256 \times 256 \times 256 \times 256$
- **Rank 8:** $16 \times 16 \times 16 \times 16 \times 16 \times 16 \times 16 \times 16$
- **Rank 16 / 32:** $2 \times 2 \times \dots \times 2$ (N dimensions of size 2)

3. Execution Matrix

For each **Tensor Rank** N in your dataset:

- Run your parallel Index Recalculation program sweeping **Core Count** (c) from 1, 2, 3, $\dots$, p .
- Record the wall-clock execution time $T(N, c)$ in seconds using high-resolution timers (e.g., `omp_get_wtime()`), averaging over 3–5 trials per data point.

Phase 3: Data Processing & Metrics Calculation

For each data point (N, c):

1. **Calculate Theoretical Memory Traffic (M):** Reading the 4 GB input array and writing to the 4 GB destination array generates 8 GB (8.589×10^9 bytes) of total memory traffic.
2. **Calculate Measured Effective Bandwidth (BW):**

$$\text{Bandwidth (GB/s)} = \frac{8.589 \text{ GB}}{\text{Execution Time } T(N, c) \text{ (seconds)}}$$

3. **Calculate Parallel Speed-up (S):**

$$S(c) = \frac{T(N, 1)}{T(N, c)}$$

Phase 4: Results Presentation (Plotting)

Populate the provided Excel template to create the final deliverable chart:

- **X-Axis:** Number of Cores used ($1 \rightarrow p$)
- **Y-Axis:** Measured Bandwidth in **GB/s**
- **Lines / Series:** One line per Tensor Rank N (e.g., Rank 2, Rank 4, Rank 8, Rank 16, Rank 32)

What to Expect & Analyze in Your Report

- **Low Ranks (e.g., Rank 2 or 4):** Inner division loops run few times (N is small). Bandwidth will scale up quickly with cores until saturating the system's physical DRAM memory bandwidth bottleneck.
- **High Ranks (e.g., Rank 16 or 32):** Inner loop overhead (performing 16+ modulo/division ops per element) becomes instruction-bound rather than memory-bound. Effective GB/s will be noticeably lower due to arithmetic overhead and cache-line striding.

User prompt: [fangmiaw@ece029 ~]\$ scp -r ./ * fangmiaw@ece029.ece.local.cmu.edu:~/647_hw2/ The authenticity of host 'ece029.ece.local.cmu.edu (172.19.136.222)' can't be established. ECDSA key fingerprint is SHA256:LBcnSQGVF0OJOCnPy/YTWZoBdT+WUEsNKoIEiwOaAuQ. Are you sure you want to continue connecting (yes/no/[fingerprint])? yes Warning: Permanently added 'ece029.ece.local.cmu.edu,172.19.136.222' (ECDSA) to the list of known hosts. scp: /afs/andrew.cmu.edu/usr18/fangmiaw/647_hw2/: No such file or directory

Response: The error scp: /afs/andrew.cmu.edu/usr18/fangmiaw/647_hw2/: No such file or directory means the destination folder ~/647_hw2/ does not exist yet on the target machine. scp cannot create missing parent or target directories automatically.

Solution

Option 1: Create the directory first, then run scp

Run mkdir over SSH to create the target directory, then copy your files:

```
ssh fangmiaw@ece029.ece.local.cmu.edu "mkdir -p ~/647_hw2"
scp -r ./ * fangmiaw@ece029.ece.local.cmu.edu:~/647_hw2/
```

Option 2: Local Copy (If you are already on ece029)

Notice that your prompt shows you are already logged into ece029: [fangmiaw@ece029 ~]\$

If you are trying to copy files **locally on the same machine**, you don't need scp or SSH network transfers at all. You can use standard Linux commands:

```
mkdir -p ~/647_hw2
cp -r ./ * ~/647_hw2/
```

Note: Copying ./ * from ~ into ~/647_hw2/ will attempt to copy everything in your home directory into a subdirectory. If you only want specific source files, navigate to that folder first or specify the files directly (e.g., cp -r my_code/* ~/647_hw2/).

User prompt: code structure for part2

Response: Here is a clean, modular **C++ / OpenMP code structure** designed to fulfill all requirements for **Part 2** (including correctness checks for all 3 algorithms and core scaling measurements for the parallelized Index Recalculation approach).

Project Directory Layout

Organize your submission repository according to the required deliverable structure:

```
Lastname-hw2/
├── src/
│   ├── main.cpp           # Main benchmarking driver & CLI entry point
│   ├── tensor.hpp         # Data structures, stride generators, & core utils
│   ├── algorithms.hpp     # Implementations of Index Recalculation, Iterative, & Recursive
│   └── Makefile           # Compilation rules with OpenMP support
├── runs/
│   └── output_log.txt     # Execution output log file
├── plots/
│   └── results.xlsx       # Filled provided Excel plot template
├── ai/
│   └── prompts.txt        # Transcripts of generative AI usage
└── README.md             # Clear instructions on how to compile and run
```

Source Code Files

1. src/tensor.hpp

Defines helper functions for stride computation, validation, and time/bandwidth metrics.

```
#ifndef TENSOR_HPP
#define TENSOR_HPP

#include <vector>
#include <iostream>
#include <numeric>
#include <cmath>

// Compute Fortran (column-major) strides: stride[k] = prod(shape[0...k-1])
inline std::vector<size_t> compute_fortran_strides(const std::vector<size_t>& shape) {
    size_t ndim = shape.size();
    std::vector<size_t> strides(ndim, 1);
    for (size_t k = 1; k < ndim; ++k) {
        strides[k] = strides[k - 1] * shape[k - 1];
    }
    return strides;
}

// Compute C (row-major) strides: stride[k] = prod(shape[k+1...N-1])
inline std::vector<size_t> compute_c_strides(const std::vector<size_t>& shape) {
    size_t ndim = shape.size();
    std::vector<size_t> strides(ndim, 1);
    if (ndim > 0) {
        for (int k = static_cast<int>(ndim) - 2; k >= 0; --k) {
            strides[k] = strides[k + 1] * shape[k + 1];
        }
    }
    return strides;
}

// Utility to verify that two flat arrays are identical
template <typename T>
bool verify_results(const std::vector<T>& ref, const std::vector<T>& test) {
    if (ref.size() != test.size()) return false;
    for (size_t i = 0; i < ref.size(); ++i) {
        if (ref[i] != test[i]) return false;
    }
    return true;
}

#endif // TENSOR_HPP
```

2. src/algorithms.hpp

Contains the **all 3 required implementations** (Sequential Index Recalculation, Iterative, Recursive) as well as the **Parallel Index Recalculation** implementation.

```
#ifndef ALGORITHMS_HPP
#define ALGORITHMS_HPP

#include "tensor.hpp"
#include <omp.h>

// -----
// 1. Index Recalculation (Sequential)
// -----

template <typename T>
void f2c_index_recalc_seq(const T* src, T* dst, size_t total_elements, const std::vector<size_t>& shape) {
    size_t ndim = shape.size();
    std::vector<size_t> strides_f = compute_fortran_strides(shape);
    std::vector<size_t> strides_c = compute_c_strides(shape);

    for (size_t flat_f = 0; flat_f < total_elements; ++flat_f) {
        size_t temp = flat_f;
        size_t flat_c = 0;

```

```

        for (int k = static_cast<int>(ndim) - 1; k >= 0; --k) {
            size_t coord = temp / strides_f[k];
            temp %= strides_f[k];
            flat_c += coord * strides_c[k];
        }
        dst[flat_c] = src[flat_f];
    }
}

// -----
// 1b. Index Recalculation (Parallel OpenMP)
// -----
template <typename T>
void f2c_index_recalc_parallel(const T* src, T* dst, size_t total_elements,
                              const std::vector<size_t>& shape, int num_threads) {
    size_t ndim = shape.size();
    std::vector<size_t> strides_f = compute_fortran_strides(shape);
    std::vector<size_t> strides_c = compute_c_strides(shape);

    #pragma omp parallel for schedule(static) num_threads(num_threads)
    for (size_t flat_f = 0; flat_f < total_elements; ++flat_f) {
        size_t temp = flat_f;
        size_t flat_c = 0;
        for (int k = static_cast<int>(ndim) - 1; k >= 0; --k) {
            size_t coord = temp / strides_f[k];
            temp %= strides_f[k];
            flat_c += coord * strides_c[k];
        }
        dst[flat_c] = src[flat_f];
    }
}

// -----
// 2. Iterative Strided Loop (Sequential)
// -----
template <typename T>
void f2c_iterative_seq(const T* src, T* dst, size_t total_elements, const std::vector<size_t>& shape) {
    size_t ndim = shape.size();
    std::vector<size_t> strides_c = compute_c_strides(shape);
    std::vector<size_t> coords(ndim, 0);
    size_t curr_c_idx = 0;

    for (size_t flat_f = 0; flat_f < total_elements; ++flat_f) {
        dst[curr_c_idx] = src[flat_f];
        for (size_t dim = 0; dim < ndim; ++dim) {
            coords[dim]++;
            curr_c_idx += strides_c[dim];
            if (coords[dim] < shape[dim]) {
                break;
            } else {
                coords[dim] = 0;
                curr_c_idx -= shape[dim] * strides_c[dim];
            }
        }
    }
}

// -----
// 3. Recursive Traversal (Sequential)
// -----
template <typename T>
void recurse_helper(const T* src, T* dst, const std::vector<size_t>& shape,
                   const std::vector<size_t>& strides_f, const std::vector<size_t>& strides_c,
                   size_t dim, size_t offset_f, size_t offset_c) {
    if (dim == shape.size()) {
        dst[offset_c] = src[offset_f];
        return;
    }
    size_t sf = strides_f[dim];
    size_t sc = strides_c[dim];
    size_t extent = shape[dim];

```

```

    for (size_t i = 0; i < extent; ++i) {
        recurse_helper(src, dst, shape, strides_f, strides_c, dim + 1, offset_f + i * sf, offset_c + i * sc);
    }
}

template <typename T>
void f2c_recursive_seq(const T* src, T* dst, size_t total_elements, const std::vector<size_t>& shape) {
    std::vector<size_t> strides_f = compute_fortran_strides(shape);
    std::vector<size_t> strides_c = compute_c_strides(shape);
    recurse_helper(src, dst, shape, strides_f, strides_c, 0, 0, 0);
}

#endif // ALGORITHMS_HPP

```

3. src/main.cpp

Drives verification, executes experiments across tensor ranks and core counts, and logs measured memory bandwidth.

```

#include "tensor.hpp"
#include "algorithms.hpp"
#include <iostream>
#include <vector>
#include <chrono>
#include <iomanip>

int main() {
    // -----
    // STEP 1: Correctness Verification on a small Tensor (Rank 3)
    // -----
    std::cout << "=== Running Correctness Check ===" << std::endl;
    std::vector<size_t> test_shape = {4, 8, 16};
    size_t test_size = 4 * 8 * 16;
    std::vector<char> src_test(test_size), dst_recalc(test_size), dst_iter(test_size), dst_recur(test_size);
    for (size_t i = 0; i < test_size; ++i) src_test[i] = static_cast<char>(i % 128);

    f2c_index_recalc_seq(src_test.data(), dst_recalc.data(), test_size, test_shape);
    f2c_iterative_seq(src_test.data(), dst_iter.data(), test_size, test_shape);
    f2c_recursive_seq(src_test.data(), dst_recur.data(), test_size, test_shape);

    bool correct = verify_results(dst_recalc, dst_iter) && verify_results(dst_recalc, dst_recur);
    std::cout << "All 3 baseline algorithms match: " << (correct ? "PASSED" : "FAILED") << "\n\n";

    // -----
    // STEP 2: Large Benchmark Experiment (4 GB = 2^32 char elements)
    // -----
    size_t N_ELEMENTS = 1ULL << 32; // 4,294,967,296 bytes (4 GB)
    std::cout << "=== Allocating 4 GB Input / Output Buffers ===" << std::endl;

    std::vector<char> src_large(N_ELEMENTS, 'A');
    std::vector<char> dst_large(N_ELEMENTS);

    // Define square / divisible tensor configurations
    struct TensorConfig {
        std::string rank_label;
        std::vector<size_t> shape;
    };

    std::vector<TensorConfig> configs = {
        {"Rank 2", {65536, 65536}},
        {"Rank 4", {256, 256, 256, 256}},
        {"Rank 8", {16, 16, 16, 16, 16, 16, 16, 16}},
        {"Rank 16", std::vector<size_t>(16, 2)},
        {"Rank 32", std::vector<size_t>(32, 2)}
    };

    int max_cores = omp_get_max_threads();
    std::cout << "Benchmarking Parallel Index Recalculation up to " << max_cores << " cores.\n\n";
}

```

```

std::cout << std::left << std::setw(10) << "Rank"
          << std::setw(10) << "Cores"
          << std::setw(15) << "Time (s)"
          << std::setw(20) << "Bandwidth (GB/s)" << std::endl;
std::cout << std::string(55, '-') << std::endl;

// Outer loop over ranks, inner loop scaling cores 1...p
for (const auto& config : configs) {
    for (int cores = 1; cores <= max_cores; ++cores) {

        // Warmup run
        f2c_index_recalc_parallel(src_large.data(), dst_large.data(), N_ELEMENTS, config.shape, cores);

        // Timed run
        double start = omp_get_wtime();
        f2c_index_recalc_parallel(src_large.data(), dst_large.data(), N_ELEMENTS, config.shape, cores);
        double elapsed = omp_get_wtime() - start;

        // Total memory traffic: 4 GB read + 4 GB write = 8.5899 GB
        double total_bytes_transferred = 2.0 * static_cast<double>(N_ELEMENTS);
        double bandwidth_gbps = (total_bytes_transferred / elapsed) / 1e9;

        std::cout << std::left << std::setw(10) << config.rank_label
                  << std::setw(10) << cores
                  << std::setw(15) << std::fixed << std::setprecision(4) << elapsed
                  << std::setw(20) << std::fixed << std::setprecision(2) << bandwidth_gbps << std::endl;
    }
    std::cout << std::string(55, '-') << std::endl;
}

return 0;
}

```

4. src/Makefile

```

CXX = g++
CXXFLAGS = -O3 -fopenmp -std=c++17 -march=native
TARGET = reorder_bench
SRCS = main.cpp

all: $(TARGET)

$(TARGET): $(SRCS)
    $(CXX) $(CXXFLAGS) $(SRCS) -o $(TARGET)

clean:
    rm -f $(TARGET)

```

Instructions to Build & Run (Include in README.md)

```

# Navigate to source directory
cd Lastname-hw2/src

# Compile using Makefile
make

# Run the benchmark binary and save logs
./reorder_bench | tee ../runs/output_log.txt

```

User prompt: change code structure, one for reorder, one for benchmark

Response: Here is the restructured code separated into two distinct programs: `reorder` (functional CLI program for correctness, testing, and single executions) and `benchmark` (automated timing driver for scaling experiments).

Project Layout

```

Lastname-hw2/
├── src/
│   ├── tensor.hpp      # Shared helper functions (strides, validation)
│   ├── algorithms.hpp  # All 3 baseline algorithms + Parallel Index Recalculation
│   ├── reorder.cpp     # Standard CLI executable (runs & verifies conversions)
│   ├── benchmark.cpp   # Automated benchmark driver (runs 4GB scaling tests)
│   └── Makefile        # Builds both binaries ('reorder' and 'benchmark')
├── runs/
│   └── output_log.txt  # Recorded execution logs
├── plots/
│   └── results.xlsx    # Provided Excel plot template
└── ai/
    └── prompts.txt     # AI transcripts/prompts list

```

Shared Headers

1. src/tensor.hpp

```

#ifndef TENSOR_HPP
#define TENSOR_HPP

#include <vector>
#include <iostream>
#include <numeric>

// Compute Fortran (column-major) strides: stride[k] = prod(shape[0...k-1])
inline std::vector<size_t> compute_fortran_strides(const std::vector<size_t>& shape) {
    size_t ndim = shape.size();
    std::vector<size_t> strides(ndim, 1);
    for (size_t k = 1; k < ndim; ++k) {
        strides[k] = strides[k - 1] * shape[k - 1];
    }
    return strides;
}

// Compute C (row-major) strides: stride[k] = prod(shape[k+1...N-1])
inline std::vector<size_t> compute_c_strides(const std::vector<size_t>& shape) {
    size_t ndim = shape.size();
    std::vector<size_t> strides(ndim, 1);
    if (ndim > 0) {
        for (int k = static_cast<int>(ndim) - 2; k >= 0; --k) {
            strides[k] = strides[k + 1] * shape[k + 1];
        }
    }
    return strides;
}

template <typename T>
bool verify_results(const std::vector<T>& ref, const std::vector<T>& test) {
    if (ref.size() != test.size()) return false;
    for (size_t i = 0; i < ref.size(); ++i) {
        if (ref[i] != test[i]) return false;
    }
    return true;
}

#endif // TENSOR_HPP

```

2. src/algorithms.hpp

```

#ifndef ALGORITHMS_HPP
#define ALGORITHMS_HPP

#include "tensor.hpp"
#include <omp.h>

// 1. Index Recalculation (Sequential)
template <typename T>
void f2c_index_recalc_seq(const T* src, T* dst, size_t total_elements, const std::vector<size_t>& shape) {

```

```

size_t ndim = shape.size();
std::vector<size_t> strides_f = compute_fortran_strides(shape);
std::vector<size_t> strides_c = compute_c_strides(shape);

for (size_t flat_f = 0; flat_f < total_elements; ++flat_f) {
    size_t temp = flat_f;
    size_t flat_c = 0;
    for (int k = static_cast<int>(ndim) - 1; k >= 0; --k) {
        size_t coord = temp / strides_f[k];
        temp %= strides_f[k];
        flat_c += coord * strides_c[k];
    }
    dst[flat_c] = src[flat_f];
}

// 1b. Index Recalculation (Parallel OpenMP)
template <typename T>
void f2c_index_recalc_parallel(const T* src, T* dst, size_t total_elements,
                              const std::vector<size_t>& shape, int num_threads) {
    size_t ndim = shape.size();
    std::vector<size_t> strides_f = compute_fortran_strides(shape);
    std::vector<size_t> strides_c = compute_c_strides(shape);

    #pragma omp parallel for schedule(static) num_threads(num_threads)
    for (size_t flat_f = 0; flat_f < total_elements; ++flat_f) {
        size_t temp = flat_f;
        size_t flat_c = 0;
        for (int k = static_cast<int>(ndim) - 1; k >= 0; --k) {
            size_t coord = temp / strides_f[k];
            temp %= strides_f[k];
            flat_c += coord * strides_c[k];
        }
        dst[flat_c] = src[flat_f];
    }
}

// 2. Iterative Strided Loop (Sequential)
template <typename T>
void f2c_iterative_seq(const T* src, T* dst, size_t total_elements, const std::vector<size_t>& shape) {
    size_t ndim = shape.size();
    std::vector<size_t> strides_c = compute_c_strides(shape);
    std::vector<size_t> coords(ndim, 0);
    size_t curr_c_idx = 0;

    for (size_t flat_f = 0; flat_f < total_elements; ++flat_f) {
        dst[curr_c_idx] = src[flat_f];
        for (size_t dim = 0; dim < ndim; ++dim) {
            coords[dim]++;
            curr_c_idx += strides_c[dim];
            if (coords[dim] < shape[dim]) {
                break;
            } else {
                coords[dim] = 0;
                curr_c_idx -= shape[dim] * strides_c[dim];
            }
        }
    }
}

// 3. Recursive Traversal (Sequential)
template <typename T>
void recurse_helper(const T* src, T* dst, const std::vector<size_t>& shape,
                   const std::vector<size_t>& strides_f, const std::vector<size_t>& strides_c,
                   size_t dim, size_t offset_f, size_t offset_c) {
    if (dim == shape.size()) {
        dst[offset_c] = src[offset_f];
        return;
    }
    size_t sf = strides_f[dim];
    size_t sc = strides_c[dim];

```

```

    size_t extent = shape[dim];

    for (size_t i = 0; i < extent; ++i) {
        recurse_helper(src, dst, shape, strides_f, strides_c, dim + 1, offset_f + i * sf, offset_c + i * sc);
    }
}

template <typename T>
void f2c_recursive_seq(const T* src, T* dst, size_t total_elements, const std::vector<size_t>& shape) {
    std::vector<size_t> strides_f = compute_fortran_strides(shape);
    std::vector<size_t> strides_c = compute_c_strides(shape);
    recurse_helper(src, dst, shape, strides_f, strides_c, 0, 0, 0);
}

#endif // ALGORITHMS_HPP

```

Executable Source Code

3. src/reorder.cpp (Functional Reordering Program)

Runs all 3 baseline approaches on a test tensor and verifies that every implementation yields identical output.

```

#include "tensor.hpp"
#include "algorithms.hpp"
#include <iostream>
#include <vector>

int main() {
    std::cout << "=== Tensor Reorder Correctness Verification ===" << std::endl;

    // Example test tensor shape [2, 3, 4]
    std::vector<size_t> shape = {2, 3, 4};
    size_t total_elements = 2 * 3 * 4;

    std::vector<char> src(total_elements);
    for (size_t i = 0; i < total_elements; ++i) {
        src[i] = static_cast<char>('A' + (i % 26));
    }

    std::vector<char> dst_recalc(total_elements);
    std::vector<char> dst_iter(total_elements);
    std::vector<char> dst_recur(total_elements);

    // Run all 3 baseline sequential implementations
    f2c_index_recalc_seq(src.data(), dst_recalc.data(), total_elements, shape);
    f2c_iterative_seq(src.data(), dst_iter.data(), total_elements, shape);
    f2c_recursive_seq(src.data(), dst_recur.data(), total_elements, shape);

    // Verify consistency across all three
    bool pass1 = verify_results(dst_recalc, dst_iter);
    bool pass2 = verify_results(dst_recalc, dst_recur);

    if (pass1 && pass2) {
        std::cout << "[SUCCESS] Index Recalculation, Iterative, and Recursive outputs match.\n";
    } else {
        std::cout << "[FAILURE] Algorithm outputs do not match!\n";
        return 1;
    }

    return 0;
}

```

4. src/benchmark.cpp (Performance & Core Scaling Benchmark Driver)

Allocates a 4 GB data buffer (2^{32} char elements), executes the parallel Index Recalculation across different ranks and core counts, and outputs measured memory bandwidth in GB/s.

```

#include "tensor.hpp"
#include "algorithms.hpp"

```

```

#include <iostream>
#include <vector>
#include <iomanip>

int main() {
    size_t N_ELEMENTS = 1ULL << 32; // 2^32 elements = 4 GB
    std::cout << "=== Allocating 4 GB Buffers (2^32 elements) ===" << std::endl;

    std::vector<char> src(N_ELEMENTS, 'X');
    std::vector<char> dst(N_ELEMENTS);

    struct TensorConfig {
        std::string rank_label;
        std::vector<size_t> shape;
    };

    std::vector<TensorConfig> configs = {
        {"Rank 2", {65536, 65536}},
        {"Rank 4", {256, 256, 256, 256}},
        {"Rank 8", {16, 16, 16, 16, 16, 16, 16, 16}},
        {"Rank 16", std::vector<size_t>(16, 2)},
        {"Rank 32", std::vector<size_t>(32, 2)}
    };

    int max_cores = omp_get_max_threads();
    std::cout << "Benchmarking Parallel Index Recalculation across 1 to " << max_cores << " cores.\n\n";

    std::cout << std::left << std::setw(10) << "Rank"
        << std::setw(10) << "Cores"
        << std::setw(15) << "Time (s)"
        << std::setw(20) << "Bandwidth (GB/s)" << std::endl;
    std::cout << std::string(55, '-') << std::endl;

    for (const auto& config : configs) {
        for (int cores = 1; cores <= max_cores; ++cores) {

            // Warmup execution
            f2c_index_recalc_parallel(src.data(), dst.data(), N_ELEMENTS, config.shape, cores);

            // Measured run
            double start = omp_get_wtime();
            f2c_index_recalc_parallel(src.data(), dst.data(), N_ELEMENTS, config.shape, cores);
            double elapsed = omp_get_wtime() - start;

            // Theoretical memory traffic: 4 GB loaded + 4 GB stored = 8.5899 GB
            double total_bytes = 2.0 * static_cast<double>(N_ELEMENTS);
            double bandwidth_gbps = (total_bytes / elapsed) / 1e9;

            std::cout << std::left << std::setw(10) << config.rank_label
                << std::setw(10) << cores
                << std::setw(15) << std::fixed << std::setprecision(4) << elapsed
                << std::setw(20) << std::fixed << std::setprecision(2) << bandwidth_gbps << std::endl;
        }
        std::cout << std::string(55, '-') << std::endl;
    }

    return 0;
}

```

5. src/Makefile

```

CXX = g++
CXXFLAGS = -O3 -fopenmp -std=c++17 -march=native

ALL_TARGETS = reorder benchmark

all: $(ALL_TARGETS)

reorder: reorder.cpp tensor.hpp algorithms.hpp
    $(CXX) $(CXXFLAGS) reorder.cpp -o reorder

```

```
benchmark: benchmark.cpp tensor.hpp algorithms.hpp
           $(CXX) $(CXXFLAGS) benchmark.cpp -o benchmark
```

```
clean:
    rm -f $(ALL_TARGETS)
```

Commands to Build and Execute

```
# 1. Compile both executables
make

# 2. Run correctness test
./reorder

# 3. Run scaling benchmarks and record log for submission
./benchmark | tee ../runs/output_log.txt
```

User prompt: code structure is complicated, it's difficult for debugging, please put tensor, algorithm.hpp code into reorder, benchmark algorithm is built for experiments required for part2

Response: Here is a streamlined, 2-file code structure. Everything is self-contained so you don't have to jump between headers during debugging:

1. `reorder.cpp`: Clean, standalone file containing all tensor helper logic and all 3 algorithm implementations (Index Recalculation, Iterative, Recursive). It runs on a small test tensor to verify correctness and print debug indices.
2. `benchmark.cpp`: Standalone file built specifically for **Part 2** scaling experiments. It allocates the 4 GB data cube (2^{32} char elements), executes the parallel Index Recalculation implementation across tensor ranks and core counts, and outputs measured bandwidth in GB/s.

Project Layout

```
Lastname-hw2/
├── src/
│   ├── reorder.cpp    # Self-contained correctness check for all 3 algorithms
│   ├── benchmark.cpp  # Self-contained Part 2 scaling & bandwidth experiment
│   └── Makefile       # Simple build script for both binaries
├── runs/
│   └── output_log.txt  # Required program execution output log
├── plots/
│   └── results.xlsx    # Provided Excel template for plots
└── ai/
    └── prompts.txt     # AI transcripts/prompts list
```

File 1: `src/reorder.cpp`

(Self-contained: Includes helper functions, all 3 algorithms, and a small sanity-check test driver)

```
#include <iostream>
#include <vector>
#include <numeric>
#include <cassert>

// =====
// HELPER FUNCTIONS
// =====

// Compute Fortran (column-major) strides: stride[k] = prod(shape[0...k-1])
std::vector<size_t> compute_fortran_strides(const std::vector<size_t>& shape) {
    size_t ndim = shape.size();
    std::vector<size_t> strides(ndim, 1);
    for (size_t k = 1; k < ndim; ++k) {
        strides[k] = strides[k - 1] * shape[k - 1];
    }
}
```

```

    }
    return strides;
}

// Compute C (row-major) strides: stride[k] = prod(shape[k+1..N-1])
std::vector<size_t> compute_c_strides(const std::vector<size_t>& shape) {
    size_t ndim = shape.size();
    std::vector<size_t> strides(ndim, 1);
    if (ndim > 0) {
        for (int k = static_cast<int>(ndim) - 2; k >= 0; --k) {
            strides[k] = strides[k + 1] * shape[k + 1];
        }
    }
    return strides;
}

// =====
// ALGORITHM 1: INDEX RECALCULATION (SEQUENTIAL)
// =====
template <typename T>
void f2c_index_recalculation(const T* src, T* dst, size_t total_elements,
                             const std::vector<size_t>& shape) {
    size_t ndim = shape.size();
    std::vector<size_t> strides_f = compute_fortran_strides(shape);
    std::vector<size_t> strides_c = compute_c_strides(shape);

    for (size_t flat_f = 0; flat_f < total_elements; ++flat_f) {
        size_t temp = flat_f;
        size_t flat_c = 0;

        // Unravel flat_f to multi-dimensional index and re-ravel to flat_c
        for (int k = static_cast<int>(ndim) - 1; k >= 0; --k) {
            size_t coord = temp / strides_f[k];
            temp %= strides_f[k];
            flat_c += coord * strides_c[k];
        }
        dst[flat_c] = src[flat_f];
    }
}

// =====
// ALGORITHM 2: ITERATIVE STRIDED LOOP (SEQUENTIAL)
// =====
template <typename T>
void f2c_iterative(const T* src, T* dst, size_t total_elements,
                   const std::vector<size_t>& shape) {
    size_t ndim = shape.size();
    std::vector<size_t> strides_c = compute_c_strides(shape);
    std::vector<size_t> coords(ndim, 0);
    size_t curr_c_idx = 0;

    for (size_t flat_f = 0; flat_f < total_elements; ++flat_f) {
        dst[curr_c_idx] = src[flat_f];

        // Increment coordinate counters
        for (size_t dim = 0; dim < ndim; ++dim) {
            coords[dim]++;
            curr_c_idx += strides_c[dim];

            if (coords[dim] < shape[dim]) {
                break;
            } else {
                coords[dim] = 0;
                curr_c_idx -= shape[dim] * strides_c[dim];
            }
        }
    }
}

// =====
// ALGORITHM 3: RECURSIVE TRAVERSAL (SEQUENTIAL)
// =====

```

```
// =====
template <typename T>
void recurse_helper(const T* src, T* dst, const std::vector<size_t>& shape,
                   const std::vector<size_t>& strides_f, const std::vector<size_t>& strides_c,
                   size_t dim, size_t offset_f, size_t offset_c) {
    if (dim == shape.size()) {
        dst[offset_c] = src[offset_f];
        return;
    }

    size_t sf = strides_f[dim];
    size_t sc = strides_c[dim];
    size_t extent = shape[dim];

    for (size_t i = 0; i < extent; ++i) {
        recurse_helper(src, dst, shape, strides_f, strides_c,
                      dim + 1, offset_f + i * sf, offset_c + i * sc);
    }
}

template <typename T>
void f2c_recursive(const T* src, T* dst, size_t total_elements,
                  const std::vector<size_t>& shape) {
    std::vector<size_t> strides_f = compute_fortran_strides(shape);
    std::vector<size_t> strides_c = compute_c_strides(shape);
    recurse_helper(src, dst, shape, strides_f, strides_c, 0, 0, 0);
}

// =====
// MAIN DEBUG DRIVER
// =====
int main() {
    std::cout << "=== Running Correctness Check for All 3 Algorithms ===" << std::endl;

    // Small tensor shape [2, 3, 4] for easy debugging
    std::vector<size_t> shape = {2, 3, 4};
    size_t total_elements = 2 * 3 * 4;

    std::vector<char> src(total_elements);
    for (size_t i = 0; i < total_elements; ++i) {
        src[i] = static_cast<char>('A' + (i % 26));
    }

    std::vector<char> dst_recalc(total_elements, 0);
    std::vector<char> dst_iter(total_elements, 0);
    std::vector<char> dst_recur(total_elements, 0);

    // Execute all three implementations
    f2c_index_recalculation(src.data(), dst_recalc.data(), total_elements, shape);
    f2c_iterative(src.data(), dst_iter.data(), total_elements, shape);
    f2c_recursive(src.data(), dst_recur.data(), total_elements, shape);

    // Check correctness
    bool pass = true;
    for (size_t i = 0; i < total_elements; ++i) {
        if (dst_recalc[i] != dst_iter[i] || dst_recalc[i] != dst_recur[i]) {
            pass = false;
            std::cerr << "[DEBUG ERROR] Mismatch at index " << i << ": "
                      << " Recalc=" << dst_recalc[i]
                      << " Iter=" << dst_iter[i]
                      << " Recur=" << dst_recur[i] << std::endl;
        }
    }

    if (pass) {
        std::cout << "[SUCCESS] All 3 algorithm implementations match perfectly!" << std::endl;
    } else {
        std::cout << "[FAILURE] Output mismatches found!" << std::endl;
        return 1;
    }
}
```

```
    return 0;
}
```

File 2: src/benchmark.cpp

(Built for Part 2 requirements: Allocates 2^{32} char elements / 4 GB, runs parallel Index Recalculation scaling tests across tensor ranks and core counts, and calculates bandwidth in GB/s)

```
#include <iostream>
#include <vector>
#include <iomanip>
#include <numeric>
#include <omp.h>

// Helper stride computations
std::vector<size_t> compute_fortran_strides(const std::vector<size_t>& shape) {
    size_t ndim = shape.size();
    std::vector<size_t> strides(ndim, 1);
    for (size_t k = 1; k < ndim; ++k) {
        strides[k] = strides[k - 1] * shape[k - 1];
    }
    return strides;
}

std::vector<size_t> compute_c_strides(const std::vector<size_t>& shape) {
    size_t ndim = shape.size();
    std::vector<size_t> strides(ndim, 1);
    if (ndim > 0) {
        for (int k = static_cast<int>(ndim) - 2; k >= 0; --k) {
            strides[k] = strides[k + 1] * shape[k + 1];
        }
    }
    return strides;
}

// Parallel Index Recalculation Implementation
template <typename T>
void f2c_index_recalc_parallel(const T* src, T* dst, size_t total_elements,
                               const std::vector<size_t>& shape, int num_threads) {
    size_t ndim = shape.size();
    std::vector<size_t> strides_f = compute_fortran_strides(shape);
    std::vector<size_t> strides_c = compute_c_strides(shape);

    #pragma omp parallel for schedule(static) num_threads(num_threads)
    for (size_t flat_f = 0; flat_f < total_elements; ++flat_f) {
        size_t temp = flat_f;
        size_t flat_c = 0;

        for (int k = static_cast<int>(ndim) - 1; k >= 0; --k) {
            size_t coord = temp / strides_f[k];
            temp %= strides_f[k];
            flat_c += coord * strides_c[k];
        }
        dst[flat_c] = src[flat_f];
    }
}

int main() {
    // Homework requirement: 2^32 (1 billion) char elements = 4 GB data cube
    size_t N_ELEMENTS = 1ULL << 32;
    std::cout << "=== Allocating 4 GB Data Cube (2^32 char elements) ===" << std::endl;

    std::vector<char> src(N_ELEMENTS, 'X');
    std::vector<char> dst(N_ELEMENTS);

    struct TensorConfig {
        std::string rank_label;
        std::vector<size_t> shape;
    };
```

```

// Square tensor shapes where total product equals 2^32
std::vector<TensorConfig> configs = {
    {"Rank 2", {65536, 65536}},
    {"Rank 4", {256, 256, 256, 256}},
    {"Rank 8", {16, 16, 16, 16, 16, 16, 16, 16}},
    {"Rank 16", std::vector<size_t>(16, 2)},
    {"Rank 32", std::vector<size_t>(32, 2)}
};

int max_cores = omp_get_max_threads();
std::cout << "Benchmarking Parallel Index Recalculation on 1 to " << max_cores << " cores...\n\n";

std::cout << std::left << std::setw(10) << "Rank"
    << std::setw(10) << "Cores"
    << std::setw(15) << "Time (s)"
    << std::setw(20) << "Bandwidth (GB/s)" << std::endl;
std::cout << std::string(55, '-') << std::endl;

for (const auto& config : configs) {
    for (int cores = 1; cores <= max_cores; ++cores) {

        // Warmup
        f2c_index_recalc_parallel(src.data(), dst.data(), N_ELEMENTS, config.shape, cores);

        // Measured run
        double start = omp_get_wtime();
        f2c_index_recalc_parallel(src.data(), dst.data(), N_ELEMENTS, config.shape, cores);
        double elapsed = omp_get_wtime() - start;

        // Total Memory Traffic = 4 GB Read + 4 GB Write = 8.5899 GB
        double total_bytes = 2.0 * static_cast<double>(N_ELEMENTS);
        double bandwidth_gbps = (total_bytes / elapsed) / 1e9;

        std::cout << std::left << std::setw(10) << config.rank_label
            << std::setw(10) << cores
            << std::setw(15) << std::fixed << std::setprecision(4) << elapsed
            << std::setw(20) << std::fixed << std::setprecision(2) << bandwidth_gbps << std::endl;
    }
    std::cout << std::string(55, '-') << std::endl;
}

return 0;
}

```

File 3: src/Makefile

```

CXX = g++
CXXFLAGS = -O3 -fopenmp -std=c++17 -march=native

all: reorder benchmark

reorder: reorder.cpp
    $(CXX) $(CXXFLAGS) reorder.cpp -o reorder

benchmark: benchmark.cpp
    $(CXX) $(CXXFLAGS) benchmark.cpp -o benchmark

clean:
    rm -f reorder benchmark

```

How to Compile and Run

1. Verify Correctness:

```

make reorder
./reorder

```

2. Run Part 2 Experiments & Save Submission Log:

```
make benchmark
./benchmark | tee ../runs/output_log.txt
```

User prompt: adhere to this example: Algorithm and implementation. A tensor is an n -dimensional data cube. To store it in memory a linearization scheme needs to be decided upon. For instance, the C language utilizes row major storage: `double A[K][M][N]` and `double B[K*M*N]` have the relationship that for `(A == B)` it holds that `A[i][j][k] = B[M*N*i + N*j + k]` for `(0 <= i < K) && (0 <= j < M) && (0 <= k < N)` The rotated cube will be represented by `double C[N][M][K]` and for `(C == B)` it holds that `C[k][j][i] = B[M*K*k + K*j + i]` for `(0 <= i < K) && (0 <= j < M) && (0 <= k < N)`

Response: Here are the updated implementations of `reorder.cpp` and `benchmark.cpp` strictly adhering to the dimension naming and formula indexing convention from your example:

- Input **Fortran (column-major) array** B_F maps coordinates $(i_0, i_1, \dots, i_{N-1})$ via:

$$B_F[\text{idx}_F] \quad \text{where} \quad \text{idx}_F = i_0 + i_1 \cdot d_0 + i_2 \cdot (d_0 \cdot d_1) + \dots$$

- Output **C (row-major) array** B_C maps rotated coordinates $(i_{N-1}, \dots, i_1, i_0)$ via:

$$B_C[\text{idx}_C] \quad \text{where} \quad \text{idx}_C = i_0 \cdot (d_1 \cdot d_2 \cdots d_{N-1}) + i_1 \cdot (d_2 \cdots d_{N-1}) + \dots + i_{N-1}$$

File 1: `src/reorder.cpp`

(Self-contained: Strides computation, Index Recalculation, Iterative, and Recursive algorithms, plus correctness testing)

```
#include <iostream>
#include <vector>
#include <cassert>

// =====
// STRIDE COMPUTATION HELPERS
// =====

// Fortran (column-major): first dimension varies fastest
// B_F index = i_0 + i_1 * d_0 + i_2 * (d_0 * d_1) + ...
std::vector<size_t> compute_fortran_strides(const std::vector<size_t>& shape) {
    size_t ndim = shape.size();
    std::vector<size_t> strides(ndim, 1);
    for (size_t k = 1; k < ndim; ++k) {
        strides[k] = strides[k - 1] * shape[k - 1];
    }
    return strides;
}

// C (row-major): last dimension varies fastest
// B_C index = i_{N-1} + i_{N-2} * d_{N-1} + i_{N-3} * (d_{N-1} * d_{N-2}) + ...
std::vector<size_t> compute_c_strides(const std::vector<size_t>& shape) {
    size_t ndim = shape.size();
    std::vector<size_t> strides(ndim, 1);
    if (ndim > 0) {
        for (int k = static_cast<int>(ndim) - 2; k >= 0; --k) {
            strides[k] = strides[k + 1] * shape[k + 1];
        }
    }
    return strides;
}

// =====
// ALGORITHM 1: INDEX RECALCULATION
// =====

template <typename T>
void f2c_index_recalculation(const T* src, T* dst, size_t total_elements,
                             const std::vector<size_t>& shape) {
```

```

size_t ndim = shape.size();
std::vector<size_t> strides_f = compute_fortran_strides(shape);
std::vector<size_t> strides_c = compute_c_strides(shape);

for (size_t flat_f = 0; flat_f < total_elements; ++flat_f) {
    size_t temp = flat_f;
    size_t flat_c = 0;

    // Unravel flat_f (Fortran) to multi-dimensional coordinate tuple (i_0, i_1, ...)
    // then calculate flat_c (C row-major) index
    for (int k = static_cast<int>(ndim) - 1; k >= 0; --k) {
        size_t coord = temp / strides_f[k];
        temp %= strides_f[k];
        flat_c += coord * strides_c[k];
    }
    dst[flat_c] = src[flat_f];
}
}

// =====
// ALGORITHM 2: ITERATIVE STRIDED LOOP
// =====
template <typename T>
void f2c_iterative(const T* src, T* dst, size_t total_elements,
                  const std::vector<size_t>& shape) {
    size_t ndim = shape.size();
    std::vector<size_t> strides_c = compute_c_strides(shape);
    std::vector<size_t> coords(ndim, 0);
    size_t curr_c_idx = 0;

    for (size_t flat_f = 0; flat_f < total_elements; ++flat_f) {
        dst[curr_c_idx] = src[flat_f];

        // Increment coordinate counters along Fortran layout
        for (size_t dim = 0; dim < ndim; ++dim) {
            coords[dim]++;
            curr_c_idx += strides_c[dim];

            if (coords[dim] < shape[dim]) {
                break;
            } else {
                coords[dim] = 0;
                curr_c_idx -= shape[dim] * strides_c[dim];
            }
        }
    }
}

// =====
// ALGORITHM 3: RECURSIVE TRAVERSAL
// =====
template <typename T>
void recurse_helper(const T* src, T* dst, const std::vector<size_t>& shape,
                  const std::vector<size_t>& strides_f, const std::vector<size_t>& strides_c,
                  size_t dim, size_t offset_f, size_t offset_c) {
    if (dim == shape.size()) {
        dst[offset_c] = src[offset_f];
        return;
    }

    size_t sf = strides_f[dim];
    size_t sc = strides_c[dim];
    size_t extent = shape[dim];

    for (size_t i = 0; i < extent; ++i) {
        recurse_helper(src, dst, shape, strides_f, strides_c,
                      dim + 1, offset_f + i * sf, offset_c + i * sc);
    }
}

template <typename T>

```

```

void f2c_recursive(const T* src, T* dst, size_t total_elements,
                  const std::vector<size_t>& shape) {
    std::vector<size_t> strides_f = compute_fortran_strides(shape);
    std::vector<size_t> strides_c = compute_c_strides(shape);
    recurse_helper(src, dst, shape, strides_f, strides_c, 0, 0, 0);
}

// =====
// MAIN DEBUG DRIVER
// =====
int main() {
    std::cout << "=== Running Correctness Check for All 3 Algorithms ===" << std::endl;

    // Tensor shape [K, M, N] = [2, 3, 4]
    std::vector<size_t> shape = {2, 3, 4};
    size_t total_elements = 2 * 3 * 4;

    std::vector<char> src(total_elements);
    for (size_t i = 0; i < total_elements; ++i) {
        src[i] = static_cast<char>('A' + (i % 26));
    }

    std::vector<char> dst_recalc(total_elements, 0);
    std::vector<char> dst_iter(total_elements, 0);
    std::vector<char> dst_recur(total_elements, 0);

    // Run all 3 implementations
    f2c_index_recalculation(src.data(), dst_recalc.data(), total_elements, shape);
    f2c_iterative(src.data(), dst_iter.data(), total_elements, shape);
    f2c_recursive(src.data(), dst_recur.data(), total_elements, shape);

    // Verify all outputs match
    bool pass = true;
    for (size_t i = 0; i < total_elements; ++i) {
        if (dst_recalc[i] != dst_iter[i] || dst_recalc[i] != dst_recur[i]) {
            pass = false;
            std::cerr << "[DEBUG ERROR] Mismatch at flat C index " << i << ": "
                      << " Recalc=" << dst_recalc[i]
                      << " Iter=" << dst_iter[i]
                      << " Recur=" << dst_recur[i] << std::endl;
        }
    }

    if (pass) {
        std::cout << "[SUCCESS] All 3 baseline algorithms match perfectly!" << std::endl;
    } else {
        std::cout << "[FAILURE] Output mismatches found!" << std::endl;
        return 1;
    }

    return 0;
}

```

File 2: src/benchmark.cpp

(Built for Part 2 Scaling Experiments: Allocates 2^{32} char elements / 4 GB, executes Parallel Index Recalculation across tensor ranks and core counts, and logs bandwidth in GB/s)

```

#include <iostream>
#include <vector>
#include <iomanip>
#include <numeric>
#include <omp.h>

std::vector<size_t> compute_fortran_strides(const std::vector<size_t>& shape) {
    size_t ndim = shape.size();
    std::vector<size_t> strides(ndim, 1);
    for (size_t k = 1; k < ndim; ++k) {
        strides[k] = strides[k - 1] * shape[k - 1];
    }
}

```

```

    return strides;
}

std::vector<size_t> compute_c_strides(const std::vector<size_t>& shape) {
    size_t ndim = shape.size();
    std::vector<size_t> strides(ndim, 1);
    if (ndim > 0) {
        for (int k = static_cast<int>(ndim) - 2; k >= 0; --k) {
            strides[k] = strides[k + 1] * shape[k + 1];
        }
    }
    return strides;
}

// Parallel Index Recalculation (OpenMP)
template <typename T>
void f2c_index_recalc_parallel(const T* src, T* dst, size_t total_elements,
                              const std::vector<size_t>& shape, int num_threads) {
    size_t ndim = shape.size();
    std::vector<size_t> strides_f = compute_fortran_strides(shape);
    std::vector<size_t> strides_c = compute_c_strides(shape);

    #pragma omp parallel for schedule(static) num_threads(num_threads)
    for (size_t flat_f = 0; flat_f < total_elements; ++flat_f) {
        size_t temp = flat_f;
        size_t flat_c = 0;

        for (int k = static_cast<int>(ndim) - 1; k >= 0; --k) {
            size_t coord = temp / strides_f[k];
            temp %= strides_f[k];
            flat_c += coord * strides_c[k];
        }
        dst[flat_c] = src[flat_f];
    }
}

int main() {
    // 2^32 (1 billion) char elements = 4 GB data cube
    size_t N_ELEMENTS = 1ULL << 32;
    std::cout << "=== Allocating 4 GB Data Cube (2^32 char elements) ===" << std::endl;

    std::vector<char> src(N_ELEMENTS, 'X');
    std::vector<char> dst(N_ELEMENTS);

    struct TensorConfig {
        std::string rank_label;
        std::vector<size_t> shape;
    };

    // Square tensor shapes whose product equals 2^32 elements
    std::vector<TensorConfig> configs = {
        {"Rank 2", {65536, 65536}},
        {"Rank 4", {256, 256, 256, 256}},
        {"Rank 8", {16, 16, 16, 16, 16, 16, 16, 16}},
        {"Rank 16", std::vector<size_t>(16, 2)},
        {"Rank 32", std::vector<size_t>(32, 2)}
    };

    int max_cores = omp_get_max_threads();
    std::cout << "Benchmarking Parallel Index Recalculation on 1 to " << max_cores << " cores...\n\n";

    std::cout << std::left << std::setw(10) << "Rank"
        << std::setw(10) << "Cores"
        << std::setw(15) << "Time (s)"
        << std::setw(20) << "Bandwidth (GB/s)" << std::endl;
    std::cout << std::string(55, '-') << std::endl;

    for (const auto& config : configs) {
        for (int cores = 1; cores <= max_cores; ++cores) {

            // Warmup execution

```

```

f2c_index_recalc_parallel(src.data(), dst.data(), N_ELEMENTS, config.shape, cores);

// Measured run
double start = omp_get_wtime();
f2c_index_recalc_parallel(src.data(), dst.data(), N_ELEMENTS, config.shape, cores);
double elapsed = omp_get_wtime() - start;

// Total memory traffic: 4 GB loaded + 4 GB stored = 8.5899 GB
double total_bytes = 2.0 * static_cast<double>(N_ELEMENTS);
double bandwidth_gbps = (total_bytes / elapsed) / 1e9;

std::cout << std::left << std::setw(10) << config.rank_label
          << std::setw(10) << cores
          << std::setw(15) << std::fixed << std::setprecision(4) << elapsed
          << std::setw(20) << std::fixed << std::setprecision(2) << bandwidth_gbps << std::endl;
}
std::cout << std::string(55, '-') << std::endl;
}

return 0;
}

```

User prompt: provide gitignore

Response: Here is a tailored `.gitignore` file for your submission directory to ensure build artifacts, binary executables, temporary compiler output, and OS junk are excluded while keeping your source code, logs, and plot deliverables clean.

Create a file named `.gitignore` in your root folder (Lastname-hw2/):

```

# Executables
src/reorder
src/benchmark
*.exe
*.out

# Object Files & Build Artifacts
*.o
*.obj
*.so
*.dylib
*.a

# OS Generated Files
.DS_Store
Thumbs.db

# IDE & Editor Files
.vscode/
.idea/
*.swp
*~

# Temporary Output Logs / Backup Files
*.tmp
*.log
!runs/*.txt

```

Note: The line `!runs/*.txt` ensures that your required experiment output log file inside the `runs/` folder is explicitly tracked and not ignored.
